

claude-windows / d4b5afb5-2f03-4b58-8952-573f1381d166

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4b5afb5-2f03-4b58-8952-573f1381d166\subagents\workflows\wf_beb40705-90b\agent-aa095dc8411c4377f.jsonl`  
- SHA-256: `8e824f1ed3777a209dd71125e3e69f40cfbc5f273d95fda4d3669e49e788dd80`  
- Source modified: `2026-06-21T03:19:55+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_beb40705-90b`  
- session_id: `d4b5afb5-2f03-4b58-8952-573f1381d166`
```

Transcript

1. user (2026-06-21T03:17:18.121Z)

You are mapping the KhelSutra web/ SPA (Vite + React 19 + TS + Tailwind 4) so it can be (a) made friendlier for non-tech-savvy users and (b) localized to Kannada + Hindi via react-i18next. Read these files fully: C:/Users/avidu/Projects/khelsutra/web/index.html and every file under C:/Users/avidu/Projects/khelsutra/web/src/ (App.tsx, main.tsx, components/{QueryBar,RallyGrid,SelectionFooter,CompileResult}.tsx, hooks/useRallySelection.ts, lib/{query,floor,format,selection,log}.ts, api/{client,types}.ts, index.css). Skip *.test.* except to note coverage. Also read C:/Users/avidu/Projects/khelsutra/web/README.md.

Produce a thorough markdown brief:

- The end-to-end USER FLOW the SPA implements (load ?video=, query bar, rally grid, selection footer, build->compile->download), step by step, as a non-tech-savvy user experiences it.
- A near-COMPLETE INVENTORY of user-facing strings (button labels, placeholders, headings, helper/error text, query-example pills, the "experimental"/"not available yet" copy, footer, the duration-first framing). Group by component. Give the count.
- Where strings are currently hardcoded (file:line) vs any existing abstraction.
- Confirm: does the SPA have ANY i18n today (no react-i18next, no t(), no catalogs)? State definitively with evidence.
- The cleanest place to wire react-i18next (provider in main.tsx/App.tsx; a locales/{en,kn,hi}/ catalog dir; a language switcher + persistence).
- UX friction a non-tech-savvy user would hit (jargon, density, small targets, desktop assumptions). Be concrete with file refs.

Return the brief as your final message. Exhaustive, faithful, file:line refs, no invention.

2. assistant (2026-06-21T03:17:20.471Z)

I'll map the KhelSutra web SPA. Let me start by pulling the repo fresh and reading all the specified files.

3. user (2026-06-21T03:17:23.826Z)

fatal: Cannot fast-forward to multiple branches.

---BRANCH---

master

4. user (2026-06-21T03:17:23.839Z)

```
1      # web/ ? KhelSutra Studio (operator console SPA)  
2  
3      Vite + React + TS + Tailwind v4. The browser UI that **drives the rally indexer  
engine**:  
4      upload ? index ? pick rallies ? reel ? download. See `docs/PER_RALLY_SELECTION.md` (the  
5      per-rally curation screen) and `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (the single-box  
appliance  
6      this SPA fronts).  
7
```

```

8      ## Dev
9      ```
10     npm install
11     npm run dev      # http://localhost:5173 ; proxies /api ? http://localhost:8000 (the
engine)
12     ```
13     Run the engine alongside it: `python -m backend.main --server` (in the badminton-
highlight-indexer
14     repo). The SPA uses **same-origin** relative `/api/*` calls ? in the appliance a reverse
proxy serves
15     this bundle and proxies `/api` to the engine on `127.0.0.1:8000`.
16
17     ## Checks (CI gates merge)
18     ```
19     npm run typecheck # tsc --noEmit
20     npm run test      # vitest run (unit tests; e.g. the typed API client)
21     npm run build     # vite build
22     ```
23
24     ## Layout
25     - `src/api/client.ts` ? typed engine client (`listRallies` / `compileReel` /
`highlightUrl` / `getConfig`),
26     grounded in the verified engine contract; every call carries a correlation id.
27     - `src/api/types.ts` ? engine data types (honest: `server`/`receiver`/`events` are
motion-only
28     fabrications and must not be surfaced).
29     - `src/lib/log.ts` ? structured client logging + correlation ids (observability seed,
appliance $5).
30     - `src/App.tsx` ? the console shell (screens land in M2+).
31

```

5. user (2026-06-21T03:17:23.852Z)

```

1      <!doctype html>
2      <html lang="en">
3      <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>KheI Sutra Studio</title>
7      </head>
8      <body>
9      <div id="root"></div>
10     <script type="module" src="/src/main.tsx"></script>
11     </body>
12     </html>
13

```

6. assistant (2026-06-21T03:17:26.643Z)

The pull failed (multiple branches config), but I'm on master. Let me note that and proceed read-only. Let me check the git state and read all the source files.

7. user (2026-06-21T03:17:28.576Z)

```

5de981f feat(site): surface "how to record reliably" on the homepage (gist of the capture
guidelines) (#39)
84f436f test(site): behavioral jsdom tests for the submit-feedback + watchable-reveal UX (#38)
a7bf087 feat(site): make the post-signup reveal watchable (manual close + no mid-watch cut-off)
(#37)
---STATUS---
?? .claude/
?? site/access.db
---WEB SRC TREE---
web/src/App.tsx

```

```

web/src/api/client.test.ts
web/src/api/client.ts
web/src/api/types.ts
web/src/components/CompileResult.tsx
web/src/components/QueryBar.examples.test.ts
web/src/components/QueryBar.tsx
web/src/components/RallyGrid.tsx
web/src/components/SelectionFooter.tsx
web/src/hooks/useRallySelection.ts
web/src/index.css
web/src/lib/floor.test.ts
web/src/lib/floor.ts
web/src/lib/format.test.ts
web/src/lib/format.ts
web/src/lib/log.ts
web/src/lib/query.test.ts
web/src/lib/query.ts
web/src/lib/selection.test.ts
web/src/lib/selection.ts
web/src/main.tsx

```

8. user (2026-06-21T03:17:28.660Z)

```

1      import { StrictMode } from "react";
2      import { createRoot } from "react-dom/client";
3      import App from "./App";
4      import "./index.css";
5
6      const root = document.getElementById("root");
7      if (!root) throw new Error("#root not found");
8
9      createRoot(root).render(
10         <StrictMode>
11           <App />
12         </StrictMode>,
13       );
14

```

9. user (2026-06-21T03:17:29.055Z)

```

1      import { useEffect, useMemo, useState } from "react";
2      import { useRallySelection } from "../hooks/useRallySelection";
3      import { RallyGrid } from "../components/RallyGrid";
4      import { QueryBar } from "../components/QueryBar";
5      import { SelectionFooter } from "../components/SelectionFooter";
6      import { CompileResult, type BuildState } from "../components/CompileResult";
7      import { runQuery, type ParsedQuery } from "../lib/query";
8      import { floorRisk } from "../lib/floor";
9      import { reelFilename } from "../lib/format";
10     import { log, newCorrelationId } from "../lib/log";
11     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
12     import type { EngineConfig, RallySegment } from "../api/types";
13
14     /** One compact, structured description of how a query was understood (for the trace
log). */
15     function parseTrace(q: ParsedQuery) {
16       return {
17         raw: q.raw,
18         recognized: q.recognized,
19         filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
20         sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
21         limit: q.limit ?? null,
22         unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
23       };

```

```

24     }
25
26     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
27     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
28     const DEMO_RALLIES: RallySegment[] = [
29         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
30         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
31         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
32         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
33         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
34         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
35     ];
36
37     type LoadState =
38         | { status: "demo" }
39         | { status: "loading" }
40         | { status: "loaded"; count: number }
41         | { status: "error"; error: string };
42
43     export default function App() {
44         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id>; absent ?
demo data.
45         const videoId = useMemo(() => new
URLSearchParams(window.location.search).get("video"), []);
46
47         const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
48         const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
49         const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
50         const [reelName, setReelName] = useState("highlights");
51         const [build, setBuild] = useState<BuildState>({ status: "idle" });
52
53         // Display order is separate from selection: a query can re-sort the cards while the
shared
54         // selection set (sel) stays the single source of truth for what lands in the reel.
55         const sel = useRallySelection(rallies, "all");
56
57         // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
58         useEffect(() => {
59             if (!videoId) return;
60             const cid = newCorrelationId();
61             log("info", "rallies.load_start", { cid, video_id: videoId });
62             let alive = true;
63             listRallies(videoId, cid)
64                 .then((rs) => {
65                     if (!alive) return;
66                     setRallies(rs);
67                     setOrder(rs);
68                     sel.apply(rs.map((r) => r.id));
69                     setLoadState({ status: "loaded", count: rs.length });
70                     log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
71                 })
72                 .catch((err) => {
73                     if (!alive) return;
74                     setLoadState({ status: "error", error: String(err) });
75                     log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
76                 });

```

```

77     return () => {
78         alive = false;
79     };
80     // videoId is stable for the page; sel.apply is a stable callback.
81     // eslint-disable-next-line react-hooks/exhaustive-deps
82     }, [videoId]);
83
84     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
85     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
86     const [cfg, setCfg] = useState<EngineConfig | null>(null);
87     useEffect(() => {
88         let alive = true;
89         getConfig()
90             .then((c) => alive && setCfg(c))
91             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
92         return () => {
93             alive = false;
94         };
95     }, []);
96     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
97
98     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
99     const ordinalOf = useMemo(() => {
100         const m = new Map<number, number>();
101         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
102         return (id: number) => m.get(id) ?? 0;
103     }, [rallies]);
104
105     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
106     const onBuild = async () => {
107         if (build.status === "building") return;
108         const cid = newCorrelationId();
109         const filename = reelFilename(reelName);
110         const ids = [...sel.selected];
111         if (!videoId) {
112             log("info", "build.no_video", { cid, selected: ids.length });
113             setBuild({
114                 status: "error",
115                 error: "Open with ?video=<video_id> against a running engine to build a reel ?
demo data can't be compiled.",
116             });
117             return;
118         }
119         setBuild({ status: "building" });
120         log("info", "build.start", {
121             cid,
122             video_id: videoId,
123             filename,
124             selected: ids.length,
125             total_sec: Math.round(sel.totalSec),
126             below_floor: warning?.atRisk.length ?? 0,
127         });
128         try {
129             const res = await compileReel(videoId, ids, filename, cid);
130             const url = highlightUrl(videoId, filename);
131             setBuild({ status: "done", url, filename });
132             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
133         } catch (err) {
134             const status = err instanceof ApiError ? err.status : undefined;
135             const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :

```

```

String(err);
136     setBuild({ status: "error", error: detail });
137     log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
138     }
139     };
140
141     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
142     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
143     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
144     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
145         const cid = newCorrelationId();
146         log("info", "query.parse", { cid, source, ...parseTrace(q) });
147
148         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
149         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
150         if (q.unavailable.length > 0) {
151             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
152         }
153
154         if (!q.recognized) {
155             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
156             return; // empty / unparseable ? leave the user's curation untouched
157         }
158
159         try {
160             const { ordered, selectedIds } = runQuery(rallies, q);
161             setOrder(ordered);
162             sel.apply(selectedIds);
163             log("info", "query.apply", {
164                 cid,
165                 source,
166                 filters: q.filters.length,
167                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
168                 limit: q.limit ?? null,
169                 total: rallies.length,
170                 selected: selectedIds.length,
171                 selected_ids: selectedIds,
172                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
173             });
174             if (selectedIds.length === 0) {
175                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
176                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
177             }
178         } catch (err) {
179             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
180             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
181         }
182     };
183
184     // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
185     const onToggle = (id: number) => {
186         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
187         sel.toggle(id);
188     };

```

```

189     const onBulk = (op: "select_all" | "clear" | "invert") => {
190         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
191         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
192         if (op === "select_all") sel.selectAll();
193         else if (op === "clear") sel.clearAll();
194         else sel.invert();
195     };
196
197     return (
198         <main className="min-h-screen bg-paper text-ink pb-24">
199             <header className="bg-ink text-white">
200                 <div className="mx-auto flex max-w-5xl items-center gap-1 px-6 py-4">
201                     <span className="text-xl font-extrabold tracking-tight">
202                         Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
203                     </span>
204                 </div>
205             </header>
206
207             <section className="mx-auto max-w-5xl px-6 py-10">
208                 <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
209                     per-rally selection · alpha
210                 </span>
211                 <h1 className="mt-4 text-3xl font-extrabold tracking-tight">Pick the rallies for
your reel</h1>
212                 <p className="mt-2 max-w-2xl text-ink/70">
213                     Filter by <strong>rally length</strong> ? e.g. <em>?top 15 longest?</em> ?
then fine-tune by tapping cards.
214                     Everything starts <strong>selected</strong>; just drop the ones you don't
want.
215                 </p>
216
217                 <ModeBanner load={loadState} videoId={videoId} />
218
219                 <div className="mt-6">
220                     <QueryBar onApply={onApplyQuery} />
221                 </div>
222
223                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
224                     <button type="button" onClick={() => onBulk("select_all")} className="rounded-
full border border-black/15 px-3 py-1 hover:border-green">Select all</button>
225                     <button type="button" onClick={() => onBulk("clear")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">Clear</button>
226                     <button type="button" onClick={() => onBulk("invert")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">Invert</button>
227                 </div>
228
229                 <div className="mt-5">
230                     <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
231                 </div>
232
233                 <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
234             </section>
235
236             <SelectionFooter
237                 count={sel.count}
238                 totalSec={sel.totalSec}
239                 reelName={reelName}
240                 onReelNameChange={setReelName}
241                 onBuild={onBuild}
242                 building={build.status === "building"}

```

```

243         warning={warning}
244     />
245 </main>
246 );
247 }
248
249 /** Pull FastAPI's {"detail": "..."} message out of an error body, else show it raw. */
250 function parseDetail(body: string): string {
251     try {
252         const j = JSON.parse(body);
253         if (j && typeof j.detail === "string") return j.detail;
254     } catch {
255         /* not JSON ? fall through */
256     }
257     return body;
258 }
259
260 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
261     if (load.status === "demo") {
262         return (
263             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
264                 Demo data ? open with <code>?video=&lt;video_id&gt;</code> against a running
engine to load a real session.
265             </p>
266         );
267     }
268     if (load.status === "loading") {
269         return (
270             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
271                 Loading rallies for <code>{videoId}</code>?
272             </p>
273         );
274     }
275     if (load.status === "error") {
276         return (
277             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
278                 Couldn't load rallies for <code>{videoId}</code>: {load.error}
279             </p>
280         );
281     }
282     return (
283         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
284             Loaded <strong className="text-ink/70">{load.count}</strong> rallies from this
session.
285         </p>
286     );
287 }
288

```

10. assistant (2026-06-21T03:17:31.446Z)

Note: there's an unrelated `site/` directory in this repo too, but the task is scoped to `web/`. Let me read all the remaining `web/src` files.

11. user (2026-06-21T03:17:32.390Z)

```

1 // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2 // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3 // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4 // text was understood, and concepts with no data behind them (shot-type / player /
score) are

```

```

5      // greyed out as "not available yet" ? never silently faked.
6      import { useMemo, useState } from "react";
7      import type { FormEvent } from "react";
8      import { parseQuery, summarize } from "../lib/query";
9      import type { ParsedQuery } from "../lib/query";
10
11      interface Props {
12          /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
13           * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
14           the trace). */
15          onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16      }
17
18      // Each example is a real, parseable query ? tapping one fills the box AND applies it.
19      // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
20      // not featured
21      // here (a test pins that every example parses to a recognized, fully-available query).
22      export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
23
24      const CHIP = "rounded-full px-2.5 py-0.5 text-xs font-medium";
25
26      export function QueryBar({ onApply }: Props) {
27          const [value, setValue] = useState("");
28          const parsed = useMemo(() => parseQuery(value), [value]);
29          const clauses = summarize(parsed);
30          const typed = value.trim().length > 0;
31
32          const applyText = (text: string) => {
33              setValue(text);
34              onApply(parseQuery(text), "example");
35          };
36
37          const submit = (e: FormEvent) => {
38              e.preventDefault();
39              if (parsed.recognized) onApply(parsed, "typed");
40          };
41
42          return (
43              <div>
44                  <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
45                      <input
46                          type="text"
47                          value={value}
48                          onChange={(e) => setValue(e.target.value)}
49                          placeholder="e.g. over 10s · top 15 longest · shortest first"
50                          aria-label="Filter rallies by length, count or order"
51                          className="min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-white
52                          px-4 py-2 text-sm text-ink outline-none focus:border-green"
53                      />
54                      <button
55                          type="submit"
56                          disabled={!parsed.recognized}
57                          className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
58                          disabled:opacity-40"
59                      >
60                          Apply
61                      </button>
62                  </form>
63
64                  { /* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
65                   */ }
66
67                  <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
68                      items-center gap-1.5">
69                      {clauses.map((c) => (

```

```

62         <span key={c} className={CHIP + " bg-green/15 text-green"}>
63             {c}
64         </span>
65     )}}
66     {parsed.unavailable.map((u) => (
67         <span
68             key={u.token}
69             title={`"${u.token}" needs ${u.kind} detection ? not available yet`}
70             className={CHIP + " bg-black/5 text-ink/45"}
71         >
72             <span className="line-through">{u.token}</span> · not available yet
73         </span>
74     )}}
75     {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
76         <span className="text-xs text-ink/50">Couldn't read that ? try ?over 10s? or
?top 15 longest?</span>
77     )}
78     {!typed && <span className="text-xs text-ink/50">Type a quick filter, or tap an
example.</span>}
79 </div>
80
81 <div className="mt-3 flex flex-wrap gap-2 text-sm">
82     <span className="text-ink/40">Try:</span>
83     {EXAMPLES.map((ex) => (
84         <button
85             key={ex}
86             type="button"
87             onClick={() => applyText(ex)}
88             className="rounded-full border border-black/15 px-3 py-1 text-xs
hover:border-green"
89         >
90             {ex}
91         </button>
92     )}}
93 </div>
94
95 <p className="mt-2 text-xs text-ink/40">
96     Filtering by <strong className="text-ink/60">rally length</strong> is fully
supported. Shot-count is{" "}
97     <span className="text-amber-700">experimental</span> (under development).
98 </p>
99 </div>
100 );
101 }
102

```

12. user (2026-06-21T03:17:32.861Z)

```

1     // The selectable rally grid ? one card per play_segment, HONEST fields only:
2     // ordinal (by start order), mm:ss start, and rally LENGTH (the trusted axis). A shot-
count chip
3     // is shown ONLY when present, and clearly flagged EXPERIMENTAL ? shot detection is
still under
4     // development and not yet reliable, so length is what we lead with (owner 2026-06-20).
5     // Never renders server/receiver/events (motion-only fabrications,
PER_RALLY_SELECTION.md §4).
6     import type { RallySegment } from "../api/types";
7     import { mmss } from "../lib/format";
8
9     interface Props {
10         rallies: RallySegment[];
11         isSelected: (id: number) => boolean;
12         onToggle: (id: number) => void;
13         /** Stable chronological label for a rally (by start_time), so re-sorting via a query
* doesn't renumber the cards. Falls back to display position when absent. */
14

```

```

15     ordinalOf?: (id: number) => number;
16   }
17
18   export function RallyGrid({ rallies, isSelected, onToggle, ordinalOf }: Props) {
19     if (rallies.length === 0) {
20       return <p className="text-ink/60 text-sm">No rallies to show yet.</p>;
21     }
22     return (
23       <ul className="grid grid-cols-[repeat(auto-fill,minmax(150px,1fr))] gap-3 list-none
p-0 m-0">
24         {rallies.map((r, i) => {
25           const selected = isSelected(r.id);
26           const n = ordinalOf ? ordinalOf(r.id) : i + 1;
27           const shot = r.metadata?.shot_count;
28           const conf = r.metadata?.shot_count_confidence;
29           return (
30             <li key={r.id}>
31               <button
32                 type="button"
33                 onClick={() => onToggle(r.id)}
34                 aria-pressed={selected}
35                 aria-label={`Rally ${n}${selected ? ", in reel" : ", not in reel"}`}
36                 className={
37                   "w-full text-left rounded-xl border p-3 transition " +
38                   (selected
39                     ? "border-green bg-green/10"
40                     : "border-black/10 bg-white hover:border-green/60")
39                 }
40             >
41               <div className="flex items-center justify-between">
42                 <span className="font-semibold text-ink">Rally {n}</span>
43                 <span
44                   aria-hidden="true"
45                   className={
46                     "inline-flex h-5 w-5 items-center justify-center rounded-full text-
xs " +
47                     (selected ? "bg-green text-[#04140D]" : "border border-black/20
text-transparent")
48                 }
49               >
50                 ?
51               </span>
52             </div>
53             <div className="mt-1 text-sm text-ink/70">
54               {mmss(r.start_time)} <span className="text-ink/40">start</span> .
<strong className="text-ink/90">{mmss(r.duration)}</strong> <span className="text-
ink/40">long</span>
55             </div>
56             {typeof shot === "number" && (
57               <div
58                 title={`Shot count is experimental (under development) and may be
inaccurate${conf && conf !== "Unknown" ? ` confidence: ${conf}` : ""}. Filter by rally length
instead.`}
59                 className="mt-2 inline-block rounded-full bg-amber-50 px-2 py-0.5
text-xs text-amber-700"
60               >
61                 ~{shot} shots . experimental
62               </div>
63             )}
64           </button>
65         </li>
66       )};
67     </ul>
68   );
69 }
70
71 );

```

```
72     }
73
```

13. user (2026-06-21T03:17:33.341Z)

```
1      // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2      // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3      // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4      // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
5      import { mmss, pluralize } from "../lib/format";
6      import type { FloorRisk } from "../lib/floor";
7
8      interface Props {
9          count: number;
10         totalSec: number;
11         reelName: string;
12         onReelNameChange: (v: string) => void;
13         onBuild: () => void;
14         /** A compile is in flight (Build wired in P5). */
15         building?: boolean;
16         /** Non-null when some selected rallies sit below the engine floor. */
17         warning: FloorRisk | null;
18     }
19
20     export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
21         const buildDisabled = count === 0 || building || warning?.allDropped === true;
22
23         return (
24             <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
25                 {warning && (
26                     <div
27                         role="status"
28                         data-testid="floor-warning"
29                         className={
30                             "border-b px-6 py-2 text-sm " +
31                             (warning.allDropped
32                                 ? "border-red-200 bg-red-50 text-red-800"
33                                 : "border-amber-200 bg-amber-50 text-amber-900")
34                     )}
35                 >
36                     {warning.allDropped ? (
37                         count === 1 ? (
38                             <>
39                                 <strong>Your only selected rally</strong> has fewer than
{warning.minShotCount} shots ? the engine
40                                 would skip it, so the build would fail. Pick a longer rally.
41                             </>
42                         ) : (
43                             <>
44                                 <strong>All {count} selected rallies</strong> have fewer than
{warning.minShotCount} shots ? the engine
45                                 skips every one, so the build would fail. Pick longer rallies.
46                             </>
47                         )
48                     ) : (
49                         <>
50                             <strong>{pluralize(warning.atRisk.length, "rally", "rallies")}</strong>
with fewer than{" "}
51                             {warning.minShotCount} shots will be skipped when building (the engine's{"
52                             ")
53                     )}
54                 )}
55             </footer>
56         )
57     }
58
```

```

52         <code>min_shot_count</code> floor).
53     </>
54     })
55 </div>
56 })
57
58 <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
59     <span className="text-sm text-ink/80" data-testid="selection-summary">
60         <strong className="text-ink">{pluralize(count, "rally", "rallies")}</strong>
selected · {mmss(totalSec)} total
61     </span>
62
63     <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
64         <span className="hidden sm:inline">Reel name</span>
65         <input
66             type="text"
67             value={reelName}
68             onChange={(e) => onReelNameChange(e.target.value)}
69             placeholder="my-highlights"
70             aria-label="Reel name"
71             className="w-40 rounded-lg border border-black/15 bg-white px-3 py-1.5 text-
ink outline-none focus:border-green"
72         />
73     </label>
74
75     <button
76         type="button"
77         onClick={onBuild}
78         disabled={buildDisabled}
79         className="rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
80     >
81         {building ? "Building?" : `Build reel (${count})`}
82     </button>
83 </div>
84 </footer>
85 );
86 }
87

```

14. user (2026-06-21T03:17:33.859Z)

```

1    // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2    // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3    // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4    // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6    export type BuildState =
7        | { status: "idle" }
8        | { status: "building" }
9        | { status: "done"; url: string; filename: string }
10       | { status: "error"; error: string };
11
12    interface Props {
13        build: BuildState;
14        onDismiss: () => void;
15    }
16
17    export function CompileResult({ build, onDismiss }: Props) {
18        if (build.status === "idle" || build.status === "building") return null;
19

```

```

20     return (
21       <section
22         data-testid="compile-result"
23         className={
24           "mt-6 rounded-xl border p-4 " +
25           (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
26         }
27       >
28         <div className="flex items-start justify-between gap-3">
29           <h2 className="font-semibold text-ink">
30             {build.status === "done" ? "Your reel is ready" : "Build failed"}
31           </h2>
32           <button
33             type="button"
34             onClick={onDismiss}
35             aria-label="Dismiss"
36             className="rounded-full px-2 text-ink/50 hover:text-ink"
37           >
38             ?
39           </button>
40         </div>
41
42         {build.status === "done" ? (
43           <div className="mt-3">
44             {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
45             <video
46               data-testid="reel-video"
47               src={build.url}
48               controls
49               playsInline
50               className="w-full max-w-xl rounded-lg bg-black"
51             />
52             <a
53               href={build.url}
54               download={build.filename}
55               className="mt-3 inline-block rounded-full bg-green px-4 py-2 text-sm font-
semibold text-[#04140D]"
56             >
57               Download {build.filename}
58             </a>
59           </div>
60         ) : (
61           <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
62             {build.error}
63           </p>
64         )}
65       </section>
66     );
67   }
68

```

15. assistant (2026-06-21T03:17:37.705Z)

Now let me read the lib, hooks, and api files.

16. user (2026-06-21T03:17:38.574Z)

```

1    // React state wrapper over the pure selection helpers in lib/selection.ts.
2    // Default = ALL selected (D8: curate-by-removal) ? the user removes warm-ups.
3    import { useCallback, useMemo, useState } from "react";
4    import type { RallySegment } from "../api/types";
5    import * as sel from "../lib/selection";
6

```

```

7      export function useRallySelection(rallies: RallySegment[], initial: "all" | "none" =
"all") {
8          const [selected, setSelected] = useState<Set<number>>(() =>
9              initial === "none" ? sel.clear() : sel.selectAll(rallies),
10          );
11
12          const toggle = useCallback((id: number) => setSelected((s) => sel.toggle(s, id)), []);
13          const selectAll = useCallback(() => setSelected(sel.selectAll(rallies)), [rallies]);
14          const clearAll = useCallback(() => setSelected(sel.clear()), []);
15          const invert = useCallback(() => setSelected((s) => sel.invert(s, rallies)),
[rallies]);
16          const isSelected = useCallback((id: number) => selected.has(id), [selected]);
17          /** Replace the whole selection (the query bar writes the shared set here ? D3). */
18          const apply = useCallback((ids: Iterable<number>) => setSelected(new Set(ids)), []);
19
20          const totalSec = useMemo(() => sel.selectedDurationSec(rallies, selected), [rallies,
selected]);
21
22          return { selected, count: selected.size, totalSec, isSelected, toggle, selectAll,
clearAll, invert, apply };
23      }
24

```

17. user (2026-06-21T03:17:39.559Z)

```

1      // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
2      // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable
axes the
3      // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4      // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5      //
6      // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7      // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8      // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9      import type { RallySegment } from "../api/types";
10
11      export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12      export type FilterField = "duration" | "shot_count";
13      /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14      export interface Filter {
15          field: FilterField;
16          op: CompareOp;
17          value: number; // seconds for duration; a raw count for shot_count
18      }
19
20      export type SortKey = "duration" | "start_time" | "shot_count";
21      export interface Sort {
22          key: SortKey;
23          dir: "asc" | "desc";
24      }
25
26      /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27      export type UnavailableKind = "shot-type" | "player" | "score";
28      export interface Unavailable {
29          token: string;
30          kind: UnavailableKind;
31      }
32
33      export interface ParsedQuery {

```

```

34     raw: string;
35     /** ANDed together. */
36     filters: Filter[];
37     sort?: Sort;
38     /** "top N" / "first N" / "N longest". */
39     limit?: number;
40     /** Concepts we understood but cannot honour (shot-type / player / score). */
41     unavailable: Unavailable[];
42     /** True when at least one actionable clause (filter | sort | limit) was understood.
43     */
44     recognized: boolean;
45 }
46 export interface QueryResult {
47     /** ALL rallies in display order (sorted if a sort was given, else input order). */
48     ordered: RallySegment[];
49     /** Ids matching the filters (then limited), in display order. */
50     selectedIds: number[];
51 }
52
53 // ?? grammar fragments ?????????????????????????????????????????????????????????????
54 // `(!\d)` stops a greedy digit-run from backtracking and leaving stray trailing digits
55 // (so "over 10 smashes" can't degrade to "over 1" + "0 smashes").
56 const NUM = "(\d+(?:\.\d+)?)(?!\\d)";
57 // Time / "shots" unit tokens. "shots?" is FIRST so it isn't eaten by the bare "s";
58 "mins?"
59 // already covers "min" (no redundant bare "min"); \b stops "m" matching "minutes" etc.
60 const UNIT_INNER = "shots?|seconds?|secs?|minutes?|mins?|m|s";
61 const UNIT = `(?:\\s*(${UNIT_INNER})\\b)?`;
62
63 // Shot-TYPE / player / score nouns ? concepts with NO detector and NO column. A number
64 // trailing
65 // one of these must NEVER become a real filter (honesty: never reroute an unsupported
66 // concept
67 // onto the duration axis ? PER_RALLY_SELECTION.md §3/§4.3). These stems are shared with
68 // the
69 // "not available yet" flagging below so the two can never drift apart.
70 const SHOT_TYPE = "smash(?:es)?|clears?|drops?|dropshots?|net
71 ?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?";
72 const PLAYER = "players?|opponents?|servers?|receivers?";
73 const SCORE = "points?|score|deuce|game point|set point|match point";
74 const UNAVAIL_WORD = `(?:${SHOT_TYPE}|${PLAYER}|${SCORE})`;
75 // Voids a comparator clause whose number is immediately followed by an unsupported
76 noun.
77 const NOT_UNAVAIL = `(?!\\s*${UNAVAIL_WORD}\\b)`;
78
79 interface OpPattern {
80     op: CompareOp;
81     re: RegExp;
82     /** Which capture group holds the number / the unit. */
83     numAt: number;
84     unitAt: number;
85 }
86
87 // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
88 // their
89 // bare counterparts ("more than" / "less than") so the negated phrase is consumed
90 // first.
91 function lead(alt: string, op: CompareOp): OpPattern {
92     return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}${NOT_UNAVAIL}`, "g"), numAt:
93 1, unitAt: 2 };
94 }
95
96 const LEADING: OpPattern[] = [
97     lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
98     lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),

```

```

89     lead("exactly|equal to|equals", "="),
90     lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
91     lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
92 ];
93
94 // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
or the
95 // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
present.
96 // NOT_UNAVAIL guards the same way ("5+ smashes" must not become duration ? 5).
97 interface TrailPattern {
98     op: CompareOp;
99     re: RegExp;
100 }
101 const POST_NOUN = "(?:\\s*(shots?)\\b)?"
102 const TRAILING: TrailPattern[] = [
103     { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater))`${POST_NOUN}``${NOT_UNAVAIL}``, "g") },
104     { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*(or
(?:less|fewer|shorter|under|below))`${POST_NOUN}``${NOT_UNAVAIL}``, "g") },
105 ];
106
107 function toSeconds(n: number, unit?: string): number {
108     return unit && /^m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
109 }
110
111 function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
112     const n = Number(numStr);
113     const isShots = !!unit && /^shots?$/i.test(unit);
114     return isShots
115         ? { field: "shot_count", op, value: n }
116         : { field: "duration", op, value: toSeconds(n, unit) };
117 }
118
119 const SORT_PATTERNS: Array<[RegExp, Sort]> = [
120     [/b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, {
key: "duration", dir: "desc" }],
121     [/b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
"asc" }],
122     [/b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
dir: "desc" }],
123     [/b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
"start_time", dir: "asc" }],
124     [/b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
desc)\b/, { key: "shot_count", dir: "desc" }],
125     [/b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
"shot_count", dir: "asc" }],
126 ];
127
128 // Order before kind so a token claimed by an earlier kind isn't double-counted. The
stems are the
129 // same SHOT_TYPE/PLAYER/SCORE used by NOT_UNAVAIL, so "flag it" and "don't filter on
it" stay in sync.
130 const UNAVAILABLE_PATTERNS: Array<[UnavailableKind, RegExp]> = [
131     ["shot-type", new RegExp(`\\b(${SHOT_TYPE})\\b`, "g")],
132     ["player", new RegExp(`\\b(${PLAYER})\\b`, "g")],
133     ["score", new RegExp(`\\b(${SCORE})\\b`, "g")],
134 ];
135
136 // A magnitude that carries a time/"shots" unit is NOT an ordinal count ("first 30
seconds" ? the
137 // first 30 rallies). This blocks the limit regexes from misreading such a number as a
count.
138 const NOT_UNIT = `(?!\\s*(?:${UNIT_INNER})\\b)`
139

```

```

140 // ?? public API ?????????????????????????????????????????????????????????????
141
142 export function parseQuery(raw: string): ParsedQuery {
143     const normalized = raw
144         .toLowerCase()
145         .replace(/?/g, ">=")
146         .replace(/?/g, "<=")
147         .replace(/(\d),(?=\d{3}\b)/g, "$1") // strip thousands separators ("1,000 shots" ?
"1000 shots")
148         .replace(/,/g, " ") // any remaining comma is a clause separator
149         .replace(/\s+/g, " ")
150         .trim();
151
152     const filters: Filter[] = [];
153     let working = normalized;
154
155     // Extract comparator clauses, blanking each match so later passes can't re-read its
number.
156     for (const pat of LEADING) {
157         working = working.replace(pat.re, (full, ...g) => {
158             filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
159             return " ".repeat(full.length);
160         });
161     }
162     for (const pat of TRAILING) {
163         working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
164             filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
165             return " ".repeat(full.length);
166         });
167     }
168
169     // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
intent).
170     let sort: Sort | undefined;
171     for (const [re, s] of SORT_PATTERNS) {
172         if (re.test(working)) {
173             sort = s;
174             break;
175         }
176     }
177
178     // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest
N"). A number
179     // carrying a unit (\b + NOT_UNIT) is NOT a count, so "first 30 seconds" / "longest 3
shots" don't
180     // become counts. `sortFromLimit` tracks a sort implied SOLELY by the limit, so if the
limit turns
181     // out invalid (e.g. "top 0") we drop that phantom sort too and the whole query reads
as a no-op.
182     let limit: number | undefined;
183     let sortFromLimit = false;
184     let m: RegExpMatchArray | null;
185     if (
186         (m = working.match(new RegExp(`\\b(\\d+)\\b${NOT_UNIT}\\s+(longest|shortest)\\b`)))
||
187         (m = working.match(new RegExp(`\\b(longest|shortest)\\s+(\\d+)\\b${NOT_UNIT}`)))
188     ) {
189         const num = /^\\d/.test(m[1]) ? m[1] : m[2];
190         const word = /^\\d/.test(m[1]) ? m[2] : m[1];
191         limit = parseInt(num, 10);
192         if (!sort) {
193             sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
194             sortFromLimit = true;
195         }
196     } else if ((m = working.match(new RegExp(`\\b(?:top|best)\\s+(\\d+)\\b${NOT_UNIT}`))))

```

```

{
197     limit = parseInt(m[1], 10);
198     if (!sort) {
199         sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
200         sortFromLimit = true;
201     }
202 } else if ((m = working.match(new
RegExp(`\\b(?:first|earliest)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
203     limit = parseInt(m[1], 10);
204     if (!sort) {
205         sort = { key: "start_time", dir: "asc" };
206         sortFromLimit = true;
207     }
208 } else if ((m = working.match(new RegExp(`\\b(?:last|latest|newest|most
recent)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
209     limit = parseInt(m[1], 10);
210     if (!sort) {
211         sort = { key: "start_time", dir: "desc" };
212         sortFromLimit = true;
213     }
214 }
215
216 // A limit must be a positive integer. Reject "top 0" (which would otherwise slice to
an EMPTY
217 // selection and silently wipe the user's curation) and clamp absurd magnitudes so the
pill never
218 // renders scientific notation.
219 if (limit != null) {
220     if (!Number.isInteger(limit) || limit < 1) {
221         limit = undefined;
222         if (sortFromLimit) sort = undefined;
223     } else if (limit > Number.MAX_SAFE_INTEGER) {
224         limit = Number.MAX_SAFE_INTEGER;
225     }
226 }
227
228 // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
229 const unavailable: Unavailable[] = [];
230 const seen = new Set<string>();
231 for (const [kind, re] of UNAVAILABLE_PATTERNS) {
232     for (const mm of normalized.matchAll(re)) {
233         const token = mm[1];
234         if (!seen.has(token)) {
235             seen.add(token);
236             unavailable.push({ token, kind });
237         }
238     }
239 }
240
241 return {
242     raw,
243     filters,
244     sort,
245     limit,
246     unavailable,
247     recognized: filters.length > 0 || sort != null || limit != null,
248 };
249 }
250
251 function rallyValue(r: RallySegment, key: SortKey): number | undefined {
252     if (key === "duration") return r.duration;
253     if (key === "start_time") return r.start_time;
254     return r.metadata?.shot_count;
255 }
256

```

```

257  /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
*/
258  export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
259    return rallies
260      .map((r, i) => ({ r, i }))
261      .sort((a, b) => {
262        const av = rallyValue(a.r, sort.key);
263        const bv = rallyValue(b.r, sort.key);
264        if (av == null && bv == null) return a.i - b.i;
265        if (av == null) return 1;
266        if (bv == null) return -1;
267        if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
268        return a.i - b.i; // tie ? preserve input order
269      })
270      .map((x) => x.r);
271  }
272
273  function matchesFilter(r: RallySegment, f: Filter): boolean {
274    const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
275    if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
276    switch (f.op) {
277      case ">":
278        return v > f.value;
279      case ">=":
280        return v >= f.value;
281      case "<":
282        return v < f.value;
283      case "<=":
284        return v <= f.value;
285      case "=":
286        return v === f.value;
287      default:
288        return false;
289    }
290  }
291
292  /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
293   * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
294   * so garbage input can never silently wipe a user's manual curation. The App also
guards on
295   * `q.recognized` before applying, so an empty box is a no-op end to end. */
296  export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
297    if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
298    const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
299    let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
300    if (q.limit != null) matched = matched.slice(0, q.limit);
301    return { ordered, selectedIds: matched.map((r) => r.id) };
302  }
303
304  // ?? human-readable labels (shared by the QueryBar pill and the tests) ?????????
305  const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
306
307  export function filterLabel(f: Filter): string {
308    if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
309    return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
310  }
311
312  export function sortLabel(s: Sort): string {
313    if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
314    if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
315    return s.dir === "desc" ? "most shots first" : "fewest shots first";

```

```

316     }
317
318     /** The count label, derived from the limit's actual sort so the pill can't contradict
    itself
319     * ("first 10", not "top 10", when the sort is oldest-first). */
320     export function limitLabel(limit: number, sort?: Sort): string {
321         if (sort?.key === "start_time") return sort.dir === "asc" ? `first ${limit}` : `last
    ${limit}`;
322         if (sort?.key === "duration" && sort.dir === "asc") return `shortest ${limit}`;
323         if (sort?.key === "shot_count") return sort.dir === "asc" ? `fewest ${limit}` : `top
    ${limit} by shots`;
324         return `top ${limit}`; // duration desc, or no sort
325     }
326
327     /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
    15"]). */
328     export function summarize(q: ParsedQuery): string[] {
329         const parts: string[] = [];
330         for (const f of q.filters) parts.push(filterLabel(f));
331         if (q.sort) parts.push(sortLabel(q.sort));
332         if (q.limit !== null) parts.push(limitLabel(q.limit, q.sort));
333         return parts;
334     }
335

```

18. user (2026-06-21T03:17:39.992Z)

```

1     // The stitching.min_shot_count floor ? surfaced, never hidden (PER_RALLY_SELECTION.md
    D7/D11).
2     // /api/compile silently drops selected rallies below the floor; the UI WARNS before
    Build so a
3     // rally never vanishes without explanation. An override is deferred (owner gate, Launch
    Report).
4     import type { RallySegment } from "../api/types";
5     import type { Selection } from "../selection";
6     import { selectedRallies } from "../selection";
7
8     export interface FloorRisk {
9         /** The configured stitching.min_shot_count the engine enforces. */
10        minShotCount: number;
11        /** Selected rallies the engine WILL drop at compile (numeric shot_count below the
        floor). */
12        atRisk: RallySegment[];
13        /** True when EVERY selected rally is below the floor ? /api/compile returns 400
        (build fails). */
14        allDropped: boolean;
15    }
16
17    /** Which SELECTED rallies the engine's min_shot_count floor would drop at compile.
18    * Mirrors backend/main.py:362-383 EXACTLY: a rally is dropped iff it has a NUMERIC
    shot_count
19    * strictly below the floor; a rally WITHOUT a shot_count is EXEMPT (kept). Returns null
    when there
20    * is nothing to warn about (no floor configured, empty selection, or none at risk). */
21    export function floorRisk(
22        rallies: RallySegment[],
23        selection: Selection,
24        minShotCount: number | undefined,
25    ): FloorRisk | null {
26        if (minShotCount == null) return null;
27        const selected = selectedRallies(rallies, selection);
28        if (selected.length === 0) return null;
29        const atRisk = selected.filter((r) => {
30            const sc = r.metadata?.shot_count;
31            return typeof sc === "number" && sc < minShotCount; // exempt when shot_count is

```

```

absent
32     });
33     if (atRisk.length === 0) return null;
34     return { minShotCount, atRisk, allDropped: atRisk.length === selected.length };
35 }
36

```

19. user (2026-06-21T03:17:40.186Z)

```

1     // Small pure formatters (node-testable, no React).
2
3     /** Seconds ? "m:ss" (e.g. 65 ? "1:05"). Rounds to the nearest second; clamps negatives
to 0. */
4     export function mmss(totalSeconds: number): string {
5         const s = Math.max(0, Math.round(totalSeconds || 0));
6         const m = Math.floor(s / 60);
7         return `${m}:${String(s % 60).padStart(2, "0")}`;
8     }
9
10    /** "3 rallies" / "1 rally" ? count with a correctly pluralized noun. */
11    export function pluralize(n: number, one: string, many = one + "s"): string {
12        return `${n} ${n === 1 ? one : many}`;
13    }
14
15    /** Turn a user reel name into a safe bare `.mp4` filename for POST /api/compile. The
engine's
16        * `safe_output_filename` (backend/utils/paths.py) rejects path separators / traversal /
absolute
17        * names, so we slugify to [a-z0-9-] and always end in `.mp4`. Empty ? "highlights.mp4".
*/
18    export function reelFilename(name: string): string {
19        const slug = (name || "")
20            .toLowerCase()
21            .replace(/^[a-z0-9-]+/g, "-")
22            .replace(/^-+|-+$/g, "")
23            .slice(0, 80);
24        return `${slug || "highlights"}.mp4`;
25    }
26

```

20. assistant (2026-06-21T03:17:42.694Z)

Now the remaining files: selection.ts, log.ts, api/client.ts, api/types.ts, index.css. And let me quickly check the test files for coverage notes and confirm no i18n anywhere.

21. user (2026-06-21T03:17:43.619Z)

```

1     // Pure selection logic over a set of rally ids (no React ? node-testable).
2     // One shared Set<segment_id> is the source of truth; the query bar, checkboxes, and
3     // (future) timeline all mutate THIS via these helpers (PER_RALLY_SELECTION.md §4).
4     import type { RallySegment } from "../api/types";
5
6     export type Selection = ReadonlySet<number>;
7
8     export const idsOf = (rallies: RallySegment[]): number[] => rallies.map((r) => r.id);
9
10    /** Toggle one id (returns a NEW set ? never mutates the input). */
11    export function toggle(sel: Selection, id: number): Set<number> {
12        const next = new Set(sel);
13        if (next.has(id)) next.delete(id);
14        else next.add(id);
15        return next;
16    }
17

```

```

18     export const selectAll = (rallies: RallySegment[]): Set<number> => new
Set(idsOf(rallies));
19     export const clear = (): Set<number> => new Set();
20
21     /** Select exactly the rallies NOT currently selected. */
22     export const invert = (sel: Selection, rallies: RallySegment[]): Set<number> =>
23         new Set(idsOf(rallies).filter((id) => !sel.has(id)));
24
25     export const selectedRallies = (rallies: RallySegment[], sel: Selection): RallySegment[]
=>
26         rallies.filter((r) => sel.has(r.id));
27
28     /** Total play-time (seconds) of the selected rallies ? what the reel will run to. */
29     export const selectedDurationSec = (rallies: RallySegment[], sel: Selection): number =>
30         selectedRallies(rallies, sel).reduce((acc, r) => acc + (r.duration || 0), 0);
31

```

22. user (2026-06-21T03:17:44.225Z)

```

1     // Structured client logger ? the observability seed for the operator console
2     // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3     // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4     // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6     export type LogLevel = "debug" | "info" | "warn" | "error";
7
8     /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
9     export function newCorrelationId(): string {
10         const c = (globalThis as { crypto?: Crypto }).crypto;
11         if (c && typeof c.randomUUID === "function") return c.randomUUID();
12         // Fallback for environments without crypto.randomUUID.
13         return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
14     }
15
16     export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
{}): void {
17         const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
});
18         if (level === "error") console.error(line);
19         else if (level === "warn") console.warn(line);
20         else console.log(line);
21     }
22

```

23. user (2026-06-21T03:17:44.573Z)

```

1     // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2     // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3     // correlation id (header + logs) so a journey is traceable end to end.
4     import { log, newCorrelationId } from "../lib/log";
5     import type { RallySegment, QueryResponse, CompileResponse, EngineConfig } from
"./types";
6
7     const BASE = "/api";
8
9     export class ApiError extends Error {
10         readonly cid?: string;
11         readonly status?: number;
12         readonly body?: string;

```

```

13     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
14         super(message);
15         this.name = "ApiError";
16         this.cid = opts.cid;
17         this.status = opts.status;
18         this.body = opts.body;
19         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20     }
21 }
22
23 interface RequestOpts {
24     method?: string;
25     body?: string;
26     headers?: Record<string, string>;
27     /** Correlation id to thread through this call; auto-generated if absent. */
28     cid?: string;
29 }
30
31 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32     const cid = opts.cid ?? newCorrelationId();
33     const url = `${BASE}${path}`;
34     const method = opts.method ?? "GET";
35     log("info", "api.request", { cid, method, url });
36
37     let res: Response;
38     try {
39         res = await fetch(url, {
40             method,
41             body: opts.body,
42             headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
43         });
44     } catch (err) {
45         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46         log("error", "api.network_error", { cid, method, url, error: String(err) });
47         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48     }
49
50     if (!res.ok) {
51         const body = await res.text().catch(() => "");
52         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54     }
55
56     const data = (await res.json()) as T;
57     log("info", "api.response", { cid, method, url, status: res.status });
58     return data;
59 }
60
61 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
62 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
63     const data = await request<QueryResponse>("/query", {
64         method: "POST",
65         body: JSON.stringify({ video_id: videoId }),
66         cid,
67     });
68     return data.play_segments ?? [];
69 }
70
71 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,

```

```

output_filename} [main.py:342]. */
72     export async function compileReel(
73         videoId: string,
74         segmentIds: number[],
75         outputFilename: string,
76         cid?: string,
77     ): Promise<CompileResponse> {
78         return request<CompileResponse>("/compile", {
79             method: "POST",
80             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename })),
81             cid,
82         });
83     }
84
85     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
86     export function highlightUrl(videoId: string, filename: string): string {
87         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
88     }
89
90     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
91     export async function getConfig(cid?: string): Promise<EngineConfig> {
92         return request<EngineConfig>("/config", { method: "GET", cid });
93     }
94
95     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
96     export function minShotCount(cfg: EngineConfig): number | undefined {
97         return cfg.stitching?.min_shot_count;
98     }
99

```

24. user (2026-06-21T03:17:45.042Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27    }

```

```

28
29     export interface QueryResponse {
30         play_segments: RallySegment[];
31     }
32
33     export interface CompileResponse {
34         status: string;
35         /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36         filepath: string;
37     }
38
39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }

```

25. user (2026-06-21T03:17:45.502Z)

```

1     @import "tailwindcss";
2
3     /* Brand tokens (mirror the marketing site palette). Tailwind v4 generates utilities
4     from these ? e.g. bg-paper, text-ink, bg-ink, text-lime, text-green. */
5     @theme {
6         --color-ink: #0c261f;
7         --color-ink2: #123a2e;
8         --color-green: #19b36b;
9         --color-lime: #c7f2b0;
10        --color-paper: #f6f8f4;
11    }
12

```

26. assistant (2026-06-21T03:17:47.904Z)

Now let me confirm definitively there's no i18n anywhere and check the package.json / test files for coverage notes.

27. user (2026-06-21T03:17:49.942Z)

No matches found

28. user (2026-06-21T03:17:50.934Z)

```

{
  "name": "kheIsutra-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "KheIsutra Studio ? operator console SPA (drives the rally indexer engine).",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  }
}

```

```

},
"devDependencies": {
  "@tailwindcss/vite": "^4.0.0",
  "@types/react": "^19.0.0",
  "@types/react-dom": "^19.0.0",
  "@vitejs/plugin-react": "^4.3.4",
  "tailwindcss": "^4.0.0",
  "typescript": "^5.7.0",
  "vite": "^6.0.0",
  "vitest": "^3.0.0"
}
}

```

29. user (2026-06-21T03:17:51.095Z)

```

1      import { describe, it, expect } from "vitest";
2      import { EXAMPLES } from "../QueryBar";
3      import { parseQuery } from "../lib/query";
4
5      // The example chips apply on tap, so a broken example would silently do nothing (or
show a
6      // "couldn't read that"). Pin that EVERY shipped example is a recognized, fully-
available query
7      // (no unsupported tokens) ? a regression guard for the duration-led example set.
8      describe("QueryBar examples", () => {
9        it("are all recognized, actionable, and fully available (no experimental/unsupported
tokens)", () => {
10          for (const ex of EXAMPLES) {
11            const q = parseQuery(ex);
12            expect(q.recognized, ex).toBe(true);
13            expect(q.unavailable, ex).toEqual([]);
14            // every example must actually constrain or order the set
15            expect(q.filters.length > 0 || q.sort !== null || q.limit !== null, ex).toBe(true);
16          }
17        });
18
19        it("lead with rally length, not the experimental shot-count axis", () => {
20          const shotExamples = EXAMPLES.filter((ex) => parseQuery(ex).filters.some((f) =>
f.field === "shot_count"));
21          expect(shotExamples).toEqual([]);
22        });
23      });
24

```

30. assistant (2026-06-21T03:17:54.832Z)

The grep confirms zero i18n. The bare ``\bt(`` pattern returned no matches, confirming no ``t()`` calls. Let me do one final confirmation check for any ``lang`/locale` handling in `main.tsx` region and check the vite config, plus verify there are no other string sources I missed (e.g., a `title/document.title`). Let me also verify the `<html lang>` is the only language marker.

31. user (2026-06-21T03:17:56.364Z)

```

C:\Users\avidu\Projects\khelsutra\web\src\components\SelectionFooter.tsx:69:
placeholder="my-highlights"
C:\Users\avidu\Projects\khelsutra\web\src\components\SelectionFooter.tsx:70:          aria-
label="Reel name"
C:\Users\avidu\Projects\khelsutra\web\src\components\RallyGrid.tsx:35:          aria-
label={`Rally ${n}${selected ? ", in reel" : ", not in reel"}`}
C:\Users\avidu\Projects\khelsutra\web\src\components\QueryBar.tsx:46:          placeholder="e.g.
over 10s · top 15 longest · shortest first"
C:\Users\avidu\Projects\khelsutra\web\src\components\QueryBar.tsx:47:          aria-
label="Filter rallies by length, count or order"
C:\Users\avidu\Projects\khelsutra\web\src\components\CompileResult.tsx:35:          aria-

```

label="Dismiss"

32. user (2026-06-21T03:17:57.662Z)

```
.
..
.gitignore
README.md
dist
index.html
node_modules
package-lock.json
package.json
src
tsconfig.json
vite.config.ts
---VITE---
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    environment: "node",
    include: ["src/**/*.test.ts"],
  },
});
```

33. assistant (2026-06-21T03:19:55.811Z)

I have everything I need. Note the test config uses `environment: "node"` and includes only `src/\*\*/\*.test.ts` ? so the `.tsx` components have no rendering tests; the brief's coverage note should reflect that. Let me confirm there are no `.test.tsx` files (App/component rendering tests) by checking the test file list once more ? from the earlier tree there were only `.test.ts` files (client, query, floor, format, selection, QueryBar.examples), no component-render tests.

I now have a complete picture. Here is the brief.

KhelSutra Studio (`web/`) ? UX + Localization Mapping Brief

Scope: `web/` SPA only (Vite + React 19 + TS + Tailwind v4). All file:line refs are to `C:/Users/avidu/Projects/khelsutra/web/...`. Repo on branch `master` at `5de981f`; `git pull --ff-only` failed (`Cannot fast-forward to multiple branches` ? local git config quirk, not a divergence I could resolve safely), so this is read against the local checkout ? flag as possibly lagging remote.

Note: this repo also contains an unrelated `site/` marketing app; it is **out of scope** and not analyzed here.

1. End-to-end USER FLOW (as a non-tech-savvy user experiences it)

The SPA is a single screen ("operator console") that drives the rally-indexer engine. There is no login, no upload, no navigation ? it is one page. (``web/README.md:1-6``; the upload/index steps happen in the engine, **before** this screen.)

****Step 0 ? How the page is reached.**** The screen is "job-tethered": a finished indexing job hands off via a URL like ``?/?video=<video_id>`` (``App.tsx:44-45``). The user does not type this ? a prior step links them here. With no ``?video=`,` the page shows built-in **\*\*demo data\*\*** (``App.tsx:28-35,47-49``).

****Step 1 ? Page loads.****

- Dark header bar: "Khel?Sutra Studio" (``App.tsx:199-205``).
- A small pill "per-rally selection · alpha" (``App.tsx:208-210``).
- Big heading: "Pick the rallies for your reel" (``App.tsx:211``).
- A one-line explainer that frames everything around ****rally length**** and ****curate-by-removal****: "Filter by ****rally length**** ? e.g. ****top 15 longest**** ? then fine-tune by tapping cards. Everything starts ****selected****; just drop the ones you don't want." (``App.tsx:212-215``).
- A ****mode banner**** under it, state-dependent (``App.tsx:217`, `ModeBanner` `App.tsx:260-287``):
 - demo ? "Demo data ? open with ``?video=<video_id>`` against a running engine to load a real session."
 - loading ? "Loading rallies for ``<videoId>``"
 - error ? "Couldn't load rallies for ``<videoId>``: `<error>`"
 - loaded ? "Loaded ****N**** rallies from this session."

****Step 2 ? Rallies load.**** If ``?video=`` is present, the SPA POSTs ``/api/query`` (``App.tsx:58-82`, `client.ts:62-69``), gets the list of rallies, and ****selects them ALL by default**** (``App.tsx:68``; "curate by removal", D8). Config (``/api/config``) is also fetched to learn the engine's ``min_shot_count`` floor (``App.tsx:86-95``).

****Step 3 ? (Optional) Query bar.**** A text box lets the user type a plain-language filter ("over 10s", "top 15 longest") (``QueryBar.tsx:42-49``). As they type, a live "parsed pill" row echoes how it was understood (``QueryBar.tsx:60-79``), and unsupported concepts (smash/player/score) are shown struck-through as "not available yet" (``QueryBar.tsx:66-74``). Pressing ****Apply**** (or tapping an example chip) re-sorts the grid and ****replaces the selection**** with what matched (``App.tsx:144-182`, `QueryBar.tsx:30-37``). Parsing is fully ****offline/deterministic**** ? no LLM, no network (``lib/query.ts:1-8``).

****Step 4 ? Hand-tune the grid.**** A grid of cards, one per rally (``RallyGrid.tsx:18-72``). Each card shows: "Rally N" (stable chronological number), the start time (``mm:ss start``), and the ****rally length**** (``mm:ss long`, bolded`) (``RallyGrid.tsx:44-57``). A shot-count chip appears only when present, flagged "~N shots · experimental" (``RallyGrid.tsx:58-65``). Tapping a card toggles it in/out of the reel (``RallyGrid.tsx:31-42`, `App.tsx:185-188``). Bulk buttons above the grid: **\*\*Select all / Clear / Invert\*\*** (``App.tsx:223-227``).

****Step 5 ? Sticky footer summary + build.**** A footer pinned to the bottom (``SelectionFooter.tsx:23-84``) shows ****N rallies**** selected · m:ss total" (``SelectionFooter.tsx:58-61``), a "Reel name" input (default "highlights"; placeholder "my-highlights") (``SelectionFooter.tsx:63-73`, `App.tsx:50``), and a green ****Build reel (N)**** button (``SelectionFooter.tsx:75-82``). If any selected rally is below the engine's shot floor, an honest ****warning banner**** appears above the bar explaining the engine will skip those rallies (or that the build will fail if all are below) (``SelectionFooter.tsx:25-56``).

****Step 6 ? Build ? compile ? download.**** Pressing Build POSTs ``/api/compile`` (``App.tsx:106-139`, `client.ts:72-83``). On demo data (no ``?video=``) it refuses with an inline error: "Open with `?video=<video_id>` against a running engine to build a reel ? demo data can't be compiled." (``App.tsx:111-118``). On success, a result panel appears (``CompileResult.tsx:17-67``): heading "Your reel is ready", an inline ``<video>`` preview, and a ****Download <filename>**** link (``CompileResult.tsx:29,42-59``). On failure: "Build failed" + the engine's reason (``CompileResult.tsx:29-31,60-64``). A ? dismisses it (``CompileResult.tsx:32-39``).

2. User-facing STRING INVENTORY (grouped by component)

All strings are **hardcoded inline in JSX/TS** ? there is no string table, constants module, or message catalog of any kind. Counts below are distinct user-visible strings (interpolated templates counted once; ``aria-label``/``title``/``placeholder`` included because they are user-facing for screen-reader / hover users).

`index.html` (1)

1. `<title>` "KhelSutra Studio" ? `index.html:6``. (Also `<html lang="en">` ? `index.html:2``, the only language marker in the app.)

`App.tsx` (15)

1. "Khel?Sutra" / "Studio" wordmark ? ``App.tsx:202`` (brand; likely keep literal).
2. "per-rally selection · alpha" pill ? ``App.tsx:209``.
3. "Pick the rallies for your reel" (h1) ? ``App.tsx:211``.
4. Explainer w/ embedded markup: "Filter by **rally length** ? e.g. **"top 15 longest"** ? then fine-tune by tapping cards. Everything starts **selected**; just drop the ones you don't want." ? ``App.tsx:212-215`` (rich text ? needs `<Trans>``).
5. "Select all" ? ``App.tsx:224``.
6. "Clear" ? ``App.tsx:225``.
7. "Invert" ? ``App.tsx:226``.
8. Build-on-demo error: "Open with `?video=<video_id>` against a running engine to build a reel ? demo data can't be compiled." ? ``App.tsx:115-116``.
9. ModeBanner demo: "Demo data ? open with `?video=<video_id>`` against a running engine to load a real session." ? ``App.tsx:262-266`` (embedded `<code>``).
10. ModeBanner loading: "Loading rallies for `<videoId>`?`" ? ``App.tsx:270-272``.
11. ModeBanner error: "Couldn't load rallies for `<videoId>`: {error}`` ? ``App.tsx:276-279``.
12. ModeBanner loaded: "Loaded **{count}** rallies from this session." ? ``App.tsx:283-285``.
13. (runtime error literal, dev-only) `"#root not found" ? `main.tsx:7`` (not user-facing in normal flow).

`QueryBar.tsx` (9)

1. placeholder "e.g. over 10s · top 15 longest · shortest first" ? ``QueryBar.tsx:46``.
2. aria-label "Filter rallies by length, count or order" ? ``QueryBar.tsx:47``.
3. "Apply" (submit button) ? ``QueryBar.tsx:56``.
4. unavailable-chip ``title``: `"{token}"` needs `{kind}` detection ? not available yet ? ``QueryBar.tsx:69``.
5. unavailable-chip body: `"? · not available yet" ? `QueryBar.tsx:72``.
6. parse-failure hint: "Couldn't read that ? try "over 10s" or "top 15 longest"." ? ``QueryBar.tsx:76``.
7. empty-state hint: "Type a quick filter, or tap an example." ? ``QueryBar.tsx:78``.
8. "Try:" label ? ``QueryBar.tsx:82``.
9. supported-vs-experimental note (rich): "Filtering by **rally length** is fully supported. Shot-count is **experimental** (under development)." ? ``QueryBar.tsx:95-98``.
- Plus the **example pills** ``["over 10s", "top 15 longest", "shortest first", "first 10"]`` ? ``QueryBar.tsx:20``. These are also the *query grammar tokens* (parsed by ``lib/query.ts``), so they are dual-purpose: display text **and** machine input ? they should generally NOT be translated (see §6).

`RallyGrid.tsx` (5)

1. empty state "No rallies to show yet." ? ``RallyGrid.tsx:20``.
2. aria-label ``Rally {n}, in reel`` / ``Rally {n}, not in reel`` ? ``RallyGrid.tsx:35``.
3. card title "Rally {n}" ? ``RallyGrid.tsx:44``.
4. card meta `"{mm:ss} start · {mm:ss} long"` (the duration-first framing; two label words "start"/"long") ? ``RallyGrid.tsx:56``.
5. shot chip ``title`` (long): "Shot count is experimental (under development) and may be inaccurate{ ? confidence: X}. Filter by rally length instead." ? ``RallyGrid.tsx:60``; and chip body `"~{shot} shots · experimental" ? `RallyGrid.tsx:63``.

`SelectionFooter.tsx` (8)

1. warning, single all-dropped: **"Your only selected rally" has fewer than {n} shots ? the engine would skip it, so the build would fail. Pick a longer rally.**" ? ``SelectionFooter.tsx:37-41``.
2. warning, many all-dropped: **"All {count} selected rallies" have fewer than {n} shots ? the engine skips every one, so the build would fail. Pick longer rallies.**" ? ``SelectionFooter.tsx:42-47``.

3. warning, partial: "***{pluralize} rally/rallies** with fewer than {n} shots will be skipped when building (the engine's `min\_shot\_count` floor)." ? `SelectionFooter.tsx:49-54`.
4. selection summary: "***{N rallies}** selected · {m:ss} total" ? `SelectionFooter.tsx:59-61` (uses `pluralize`).
5. "Reel name" label ? `SelectionFooter.tsx:64`.
6. placeholder "my-highlights" ? `SelectionFooter.tsx:69`.
7. aria-label "Reel name" ? `SelectionFooter.tsx:70`.
8. button "Building?" / "Build reel ({count})" ? `SelectionFooter.tsx:81`.

`CompileResult.tsx` (4)

1. heading "Your reel is ready" / "Build failed" ? `CompileResult.tsx:29-31`.
2. aria-label "Dismiss" + ? glyph ? `CompileResult.tsx:35-38`.
3. "Download {filename}" link ? `CompileResult.tsx:57`.
4. error body ? pass-through of `build.error` (engine text) ? `CompileResult.tsx:61-63`.

`lib/format.ts` (pluralization helper, not strings)

- `pluralize(n, "rally", "rallies")` ? call sites pass the English noun forms (`SelectionFooter.tsx:50,60`). The helper's English-only `+ "s"` default (`format.ts:11-13`) is ****not i18n-safe**** for Kannada/Hindi plural rules (see §6).

Pure-logic label strings (machine-derived, surfaced in the query pill)

`lib/query.ts` builds human pill text: `filterLabel` ("length ? 10s", "shots = 5")
`query.ts:307-310`; `sortLabel` ("longest first", "shortest first", "newest first", "oldest first", "most shots first", "fewest shots first") `query.ts:312-316`; `limitLabel` ("first 10", "last 10", "shortest 10", "top 10", "top 10 by shots", "fewest 10") `query.ts:320-325`. These render in the green pills via `summarize()` (`QueryBar.tsx:27,61-65`). They are user-facing but derived in the lib layer ? localizing them requires either moving formatting up to the component or injecting a `t` function into `lib/query.ts`.

****Total user-facing strings: ~46**** across components (App 15, QueryBar 9 + 4 example pills, RallyGrid 5, SelectionFooter 8, CompileResult 4, index.html 1), ****plus ~16 derived pill labels**** in `lib/query.ts`. Call it ****~50 distinct UI strings + ~16 derived labels = ~66 message keys**** for a full catalog (excluding brand wordmark and the dev-only `#root not found`).

3. Where strings live: hardcoded vs abstraction

****Everything is hardcoded inline.**** There is ****no abstraction layer**** for copy:

- No constants file, no `messages.ts`, no `strings/` dir.
- The only existing "shared text" abstractions are ***behavioral helpers***, not catalogs:
 - `EXAMPLES` array in `QueryBar.tsx:20` (shared with a test).
 - `pluralize` in `format.ts:11-13` (English plural logic baked in).
 - The pill-label formatters in `query.ts:305-334` (`OP\_SYMBOL`, `filterLabel`, `sortLabel`, `limitLabel`, `summarize`).
- Several strings carry ****embedded JSX markup**** (`<strong>`, `<em>`, `<code>`, `<span class>`):
`App.tsx:212-215`, `App.tsx:262-285`, `QueryBar.tsx:95-98`, `RallyGrid.tsx:56`,
`SelectionFooter.tsx:37-54,59-61`. These cannot be flat-string-replaced; they need
`<Trans>-style interpolation.
- Several strings carry ****runtime interpolation**** (`{videoId}`, `{count}`, `{n}`, `{mm:ss}`,
{filename}`, `{token}/{kind}`): same locations plus `RallyGrid.tsx:35,60`,
`CompileResult.tsx:57`.

4. Does the SPA have ANY i18n today?

****Definitively NO.**** Evidence:

- `package.json` dependencies are `react` + `react-dom` only; devDeps are build/test tooling.
- **No `react-i18next`, `i18next`, `react-intl`, `@lingui`, `formatjs`**** (`package.json:13-26`).
- A repo-wide grep for `i18n|i18next|useTranslation|\\b(|locale|IntlProvider|react-intl|formatjs|lingui` over `web/` returned ****No matches found****.
- A grep for `lang|navigator.|localStorage|document.title` found ****zero**** locale detection,

language persistence, or runtime `document.title` setting ? the only language declaration is the static `<html lang="en">` (`index.html:2`).

- No `locales/`, `messages/`, or catalog directory exists under `web/src` (full tree confirmed: 21 files, all `.tsx/.ts/.css`).
- `main.tsx:1-13` renders `<App/>` directly with **no provider wrapper** of any kind.

The app is **100% hardcoded English**, single-locale.

5. Cleanest place to wire react-i18next

The architecture is small and clean, which makes wiring straightforward:

Provider. Add the i18n init + provider at the **app root** in `main.tsx` (`main.tsx:9-13`). Either import a side-effecting `./i18n.ts` (calls `i18n.use(initReactI18next).init(...)`) before render, or wrap `<App/>` in `<I18nextProvider i18n={i18n}>`. `main.tsx` is the single render entry, so this is the one place that guarantees the context wraps everything. No other provider exists to coexist with.

Catalog dir. Create `web/src/locales/{en,kn,hi}/common.json` (or split by namespace: `app`, `query`, `grid`, `footer`, `result`). Keys mirror `$2`'s grouping. Vite imports JSON natively, so static `import en from './locales/en/common.json'` in `i18n.ts` works without extra tooling (no backend/HTTP loader needed for an offline appliance).

Derived labels. `lib/query.ts` `filterLabel`/`sortLabel`/`limitLabel`/`summarize` (`query.ts:305-334`) are pure and must stay test-friendly (node env, `query.test.ts`). Cleanest: keep them returning **structured descriptors** (they nearly do ? `{field, op, value}`, `{key, dir}`, `limit`) and move the `*text*` rendering into the `QueryBar` component using `t()` with interpolation, OR pass a `t` formatter into `summarize`. Do NOT scatter `useTranslation` into the lib layer (breaks the node-only test env in `vite.config.ts:test.environment="node"`).

Language switcher + persistence. Add a small `<select>` in the header (`App.tsx:200-204`, next to the wordmark) calling `i18n.changeLanguage(lng)`. Persist via `i18next-browser-languagedetector` with order `['localStorage', 'navigator', 'htmlTag']` and `caches: ['localStorage']` ? there is currently zero `localStorage` usage, so no collision. Also set `<html lang>` reactively on change (today it is hardcoded `en` at `index.html:2`) for a11y/SEO correctness.

Pluralization. Replace `format.ts pluralize` (`format.ts:11-13`) at its two call sites (`SelectionFooter.tsx:50,60`) with i18next's plural support (`t('footer.summary', { count })` + _one`/_other`/_many` keys), since Hindi/Kannada plural rules differ from English +'s'.`

Number/time. `mmss` (`format.ts:4-8`) is locale-neutral (`m:ss`) and can stay; but Kannada/Hindi may want native-digit rendering via `Intl.NumberFormat` if the owner wants full localization.

6. UX friction for a non-tech-savvy user (concrete)

Engineering/internal jargon leaks into the UI:

- `?video=<video_id>` URL/placeholder language is shown verbatim to users in **two** banners and an error (`App.tsx:115-116`, `App.tsx:264`, demo/error banners `App.tsx:262-279`). A non-technical user cannot act on "open with `?video=<video_id>` against a running engine".
- "engine", "compile", "min_shot_count floor", `<code>min_shot_count</code>` surface raw backend vocabulary in the footer warning (`SelectionFooter.tsx:51-53`) and throughout. "the engine's `min_shot_count` floor`" is meaningless to an end user.
- "alpha", "experimental", "(under development)" appear in multiple places (`App.tsx:209`, `QueryBar.tsx:97`, `RallyGrid.tsx:60-63`). Honest, but anxiety-inducing and unexplained for a lay user.
- "reel", "rally", "stitched", "compile" assume domain fluency. "Reel name" (`SelectionFooter.tsx:64`) with placeholder "my-highlights" mixes two metaphors.
- Build-failure surfaces **raw engine error text** straight through (`CompileResult.tsx:60-64`,

fed by `parseDetail` of FastAPI `{detail}` at `App.tsx:135,250-258`) ? likely a stack-ish or technical message, untranslated and unfriendly.

****The query bar over-trusts text literacy:****

- The whole primary filter mechanism is a ****free-text command line**** ("over 10s", "top 15 longest") (`QueryBar.tsx:42-49`). A non-tech-savvy and/or non-English user will struggle to type valid English grammar tokens. The deterministic parser only understands ****English**** keywords (`query.ts:59-134`) ? a Hindi/Kannada user typing native words gets "Couldn't read that" (`QueryBar.tsx:76`). ****Critical localization gap:**** translating the UI chrome does NOT make the query bar work in Kannada/Hindi; the grammar itself is English-only. Consider replacing/augmenting the text bar with ****tap-only controls**** (sort dropdown, length slider, the existing example chips promoted to primary).
- The example pills double as machine input (`QueryBar.tsx:20`), so they can't simply be translated ? a Kannada label that no longer parses would break tap-to-apply (guarded by `QueryBar.examples.test.ts`). This is a real i18n tension to design around.

****Density / hierarchy:****

- The parsed-pill row, "not available yet" struck-through chips, "Try:" chips, and the "experimental" note (`QueryBar.tsx:60-98`) stack a lot of meta-information under one input ? high cognitive load for a novice.
- Two separate "experimental/not-available" treatments (greyed strike-through chips vs amber note vs amber card chips) may read as errors/warnings to a lay user.

****Small touch targets / desktop assumptions:****

- Bulk buttons, example chips, and unavailable chips use `px-3 py-1` / `py-0.5` / `text-xs` (`App.tsx:224-226`, `QueryBar.tsx:62-92`) ? well under the ~44px touch-target guideline; hard for thumbs on mobile.
- The card grid is `minmax(150px,1fr)` (`RallyGrid.tsx:23`); the whole layout is centered in `max-w-5xl` columns (`App.tsx:200,207`, `SelectionFooter.tsx:58`) ? a desktop-console layout. On a phone the `min-w-[16rem]` input (`QueryBar.tsx:48`) and sticky footer with many inline items (`SelectionFooter.tsx:58`) will wrap awkwardly.
- "Reel name" label is ****hidden on small screens**** (`hidden sm:inline`, `SelectionFooter.tsx:64`), leaving a bare input with only a placeholder ? a novice may not know what it's for.
- Hover-only affordances: card hover borders (`RallyGrid.tsx:40`), and key explanations live in `title` tooltips (`QueryBar.tsx:69`, `RallyGrid.tsx:60`) that don't exist on touch devices ? so the ***reason*** a chip says "not available yet" or that shot-count is unreliable is ****invisible on mobile****.

****Selection model surprise:****

- "Everything starts selected; drop what you don't want" (`App.tsx:213-215`, default-all at `App.tsx:55,68`, `useRallySelection.ts:7-9`) is a deliberate but non-obvious inversion of the usual "pick what you want." Applying a query ***replaces*** the whole selection (`App.tsx:160-162`, `useRallySelection.ts:18`), which can silently discard hand-tuning ? a novice may not connect Apply with "my checkmarks changed."
- The ?/empty circle on cards is `aria-hidden` and color-only (green fill vs transparent) (`RallyGrid.tsx:45-53`) ? selection state leans on color, a contrast/colorblind concern.

****Positive UX notes (keep):**** the duration-first framing is consistent and honest

(`App.tsx:212-215`, `RallyGrid.tsx:56`, `QueryBar.tsx:95-98`); the floor warning is genuinely user-protective (`SelectionFooter.tsx:25-56`); aria-labels exist on most interactive elements (`QueryBar.tsx:47`, `RallyGrid.tsx:35`, `SelectionFooter.tsx:70`, `CompileResult.tsx:35`).

7. Test coverage note (for context)

`vite.config.ts` sets `test.environment: "node"` and `include: ["src/\*\*/\*.test.ts"]` ? so ****only pure `.ts` logic is tested**** (`api/client.test.ts`, `lib/{query,floor,format,selection}.test.ts`, `components/QueryBar.examples.test.ts`). There are ****no `.test.tsx` component-render tests**** ? `App.tsx`, `QueryBar.tsx`, `RallyGrid.tsx`, `SelectionFooter.tsx`, `CompileResult.tsx` rendering/copy is ****untested****. Practical implication: a refactor to wire i18n (replacing literals with `t()` and a provider) has ****no rendering safety net**** in `web/` today ? the i18n work should add a jsdom test env + a few

render tests (the sibling `site/` app already does behavioral jsdom tests per commit `84f436f`, a usable pattern to copy). `QueryBar.examples.test.ts` is the one guard that pins example-pill tokens parse correctly ? keep it green when localizing pills.

****Key files for the i18n/UX work:**** `web/src/main.tsx` (provider), `web/src/App.tsx`,
`web/src/components/{QueryBar,RallyGrid,SelectionFooter,CompileResult}.tsx` (string sites),
`web/src/lib/{query.ts,format.ts}` (derived labels + plural), `web/index.html` (`lang`),
`web/package.json` + `web/vite.config.ts` (add deps + jsdom test env).